

Student Resource Guide

Generative AI for Beginners

Section 6: Training and Fine-Tuning | Lecture 6.2: Fine-Tuning Explained

Learning Objectives

- Explain why fine-tuning is needed and what it adds beyond pre-training
- Describe the difference between full fine-tuning, LoRA, and RLHF approaches
- Evaluate the trade-offs between fine-tuning cost, data requirements, and performance gains
- Identify use cases where fine-tuning is the correct solution vs. prompt engineering or RAG
- Assess the risks of catastrophic forgetting and how to mitigate them during fine-tuning

Introduction

Fine-tuning is the bridge between a pre-trained model with broad general capabilities and a production-ready AI system optimised for a specific task, domain, or behaviour. Where pre-training gives the model knowledge and language understanding, fine-tuning gives it purpose: the ability to follow instructions helpfully, respond consistently in a specific domain's terminology, or produce outputs in a defined format. Understanding fine-tuning is essential for practitioners making decisions about when to fine-tune, how to fine-tune, and how to evaluate whether fine-tuning has succeeded.

The economics of fine-tuning are dramatically more accessible than pre-training. While training GPT-4 from scratch costs tens of millions of dollars, fine-tuning a capable open-weight model on domain-specific data can cost hundreds to low thousands of dollars using consumer cloud GPU resources. This accessibility has democratised the ability to create specialised AI systems, enabling organisations of all sizes to build task-specific models rather than relying entirely on general-purpose commercial APIs.

However, fine-tuning introduces its own failure modes: catastrophic forgetting (the model loses general capabilities), overfitting to the fine-tuning distribution, and unintended behavioural changes. Practitioners need to understand these risks and the mitigation approaches—low learning rates, regularisation, parameter-efficient fine-tuning methods, and comprehensive evaluation—to execute fine-tuning projects successfully.

Key Concepts

Full Fine-Tuning

Definition	A fine-tuning approach that updates all parameters of the pre-trained model on a task-specific dataset. The entire model is trained end-to-end with a low learning rate to preserve pre-trained knowledge while adapting to the new task. Requires significant GPU memory (the full model must fit in VRAM) and compute.
Business Context	Full fine-tuning produces the best possible task-specific performance because every parameter can adapt. However, it requires storing and serving a separate model copy per use case, and risks catastrophic forgetting if the learning rate is too high or training continues too long. For organisations with a small number of specialised use cases and adequate compute, full fine-tuning is appropriate.
AI Practitioner relevance	Practitioners choosing between full fine-tuning and parameter-efficient alternatives should consider: GPU memory availability (full fine-tuning requires storing the model in training mode—2–4x inference memory), number of specialised use cases (one per full fine-tune), and acceptable performance ceiling (full fine-tuning has the highest ceiling).
Real-World Example	Bloomberg trained BloombergGPT (50B parameter model) with full fine-tuning on 700B tokens of financial data. The model significantly outperforms general models on financial NLP tasks—but at the cost of pre-training-level compute and the maintenance of a completely separate model from the general-purpose base.
Key Risks	Catastrophic forgetting: with too high a learning rate or too long training on narrow data, the model may lose general capabilities (language understanding, reasoning) in favour of task-specific pattern matching.

Parameter-Efficient Fine-Tuning (PEFT) and LoRA

Definition	Fine-tuning approaches that update only a
-------------------	---

	small fraction of model parameters while freezing the rest, dramatically reducing compute and memory requirements. LoRA (Low-Rank Adaptation) adds small trainable rank-decomposition matrices to the model's weight matrices, training only these adapters. Typically 0.1–1% of parameters are trained.
Business Context	LoRA enables fine-tuning large models on consumer-grade hardware (a single 24GB GPU can fine-tune Llama 3 70B with LoRA via quantisation). Multiple LoRA adapters for different use cases can be applied to the same base model at inference time, enabling multi-use-case deployment without separate model copies.
AI Practitioner relevance	LoRA is the standard starting point for practitioners conducting fine-tuning projects. The rank parameter (r) controls the capacity of the LoRA adapters: higher rank = more parameters, more task-specific expressiveness, but more memory. Typical values: $r=4$ for style/tone adjustments, $r=16$ – 64 for domain knowledge adaptation, $r=128+$ for complex reasoning tasks.
Real-World Example	Stable Diffusion's LoRA fine-tuning ecosystem demonstrates the power of PEFT: thousands of community-created LoRA adapters exist for different art styles, character designs, and photographic aesthetics—each only a few MB in size, loadable on top of the base model without requiring separate model deployment.

Instruction Tuning and RLHF

Definition	Instruction tuning is supervised fine-tuning on instruction-response pairs to teach the model to follow natural language instructions. RLHF (Reinforcement Learning from Human Feedback) is a further fine-tuning stage that uses human preference ratings between model outputs to train a reward model, then uses this reward model to further fine-tune the language model via RL—aligning outputs with human values and preferences.
Business Context	Instruction tuning and RLHF are why commercial models like ChatGPT and

	Claude are helpful, harmless, and honest rather than merely capable text predictors. Without these stages, a pre-trained model responds as a text completion system—unhelpfully continuing inputs rather than answering questions. RLHF is the primary mechanism for AI alignment in production systems.
AI Practitioner relevance	Practitioners assessing whether a foundation model is appropriate for a user-facing application should distinguish between base models (pre-trained only) and instruction-tuned models (with SFT and/or RLHF). Base models are not suitable for direct user interaction. Instruction-tuned variants of the same model are typically what practitioners should select for deployed applications.
Real-World Example	Llama 3 base model and Llama 3 Instruct are pre-trained on identical data. Llama 3 base continues text—given 'The capital of France is', it continues '...Paris, which is located in northern France...'. Llama 3 Instruct, after instruction tuning, responds to 'What is the capital of France?' with 'The capital of France is Paris.' The instruction tuning fundamentally changes the interaction paradigm.

Catastrophic Forgetting

Definition	A failure mode in fine-tuning where training on a narrow task-specific dataset causes the model to lose performance on general capabilities it demonstrated before fine-tuning. Occurs when the fine-tuning gradient updates overwrite the pre-trained weights that encoded general knowledge.
Business Context	Catastrophic forgetting can make a fine-tuned model worse on general tasks even as it improves on the target task. This is a problem when the deployed system needs to handle both the specific fine-tuned task and general queries—a common real-world scenario. Prevention requires careful hyperparameter selection and evaluation across both task-specific and general benchmarks.
AI Practitioner relevance	To mitigate catastrophic forgetting: use low

	learning rates (1e-5 to 5e-5 for most fine-tuning), prefer PEFT/LoRA (which freezes base model weights), include a diverse data replay component (a small portion of general pre-training data mixed into fine-tuning), and always evaluate on general benchmarks alongside task-specific ones after fine-tuning.
Real-World Example	A team fine-tuned a general-purpose LLM on customer service conversations for a retail brand. Post-fine-tuning evaluation showed 15% performance improvement on customer service tasks but 12% degradation on general language understanding benchmarks—indicating catastrophic forgetting. Resolution: reduce learning rate by 5x, add 10% general data to fine-tuning mix, and retrain.

Real-World Applications

Fine-tuning is the primary mechanism for adapting foundation models to specific commercial use cases across industries.

Domain-Specific Customer Service AI: Organisations fine-tune general LLMs on their product documentation, support ticket history, and company-specific terminology to create customer service AI that understands their specific products, policies, and customer language patterns. Fine-tuning on 5,000–50,000 historical support interactions typically produces substantial improvement over prompt-engineering-only approaches for domain-specific query handling.

Code Generation for Specific Frameworks: Development tools fine-tune general coding models on company-specific codebases, internal libraries, and proprietary APIs to create code assistants that understand and generate code using internal conventions. GitHub Copilot Enterprise allows organisations to fine-tune on their private repositories for this purpose.

Tone and Style Adaptation for Brand Communications: Marketing and communications teams fine-tune language models on samples of brand-approved content to create AI writing assistants that naturally produce outputs matching the brand's voice, terminology preferences, and style guidelines—without requiring extensive prompting for each generation.

Medical and Clinical AI Applications: Healthcare organisations fine-tune models on medical literature, clinical notes (appropriately de-identified), and diagnostic guidelines to create AI systems that perform well on clinical NLP tasks—entity extraction,

summarisation, coding, and decision support—while maintaining general language capability.

Best Practices

Start with the smallest fine-tuning intervention that achieves your target performance. Try prompt engineering and RAG before fine-tuning. Then try LoRA before full fine-tuning. Escalate compute and complexity only when demonstrated necessary.

Use a learning rate at least 10x lower than pre-training. Fine-tuning should gently adjust pre-trained weights, not overwrite them. Start with $1e-5$ and adjust based on training dynamics. High learning rates are the most common cause of catastrophic forgetting.

Maintain a held-out general evaluation set alongside task-specific evaluation. Track performance on general capabilities throughout fine-tuning to detect catastrophic forgetting early. Do not rely solely on task-specific metrics.

Curate your fine-tuning data quality aggressively. Fine-tuning data quality matters more than quantity. 1,000 high-quality, diverse, representative examples often outperform 100,000 noisy examples. Invest proportionally more time in data curation than in hyperparameter tuning.

Version control your fine-tuning data and checkpoints. Treat fine-tuning as a repeatable engineering process. Version your datasets, document preprocessing steps, and save intermediate checkpoints. This enables reproducing results and systematically iterating on data quality.

Common Mistakes to Avoid

Mistake: Fine-tuning when RAG or prompt engineering would suffice.

Correct approach: Fine-tuning is appropriate when task-specific behaviour cannot be reliably achieved through prompting, when latency is a constraint (fine-tuning can reduce prompt length), or when consistent formatting is critical. For knowledge retrieval tasks, RAG is almost always the correct first approach.

Mistake: Using the instruction-tuned model for fine-tuning when the goal is domain knowledge adaptation.

Correct approach: If you are adding domain knowledge and the instruction-following behaviour is already correct, fine-tune on top of the instruction-tuned variant to preserve alignment. If you are adapting base model behaviour at a fundamental level, consider starting from the base model. The choice depends on whether you need to preserve or override existing instruction-following behaviour.

Mistake: Evaluating fine-tuning success only on the training distribution.

Correct approach: Fine-tuned models can overfit to training examples or their formatting. Always evaluate on a held-out test set drawn from the same distribution as production inputs. Additionally, test on edge cases and out-of-distribution inputs to assess generalisation.

Practitioner Tool: Fine-Tuning vs. Alternatives Decision Framework

Type: Decision Framework

Use this framework to determine whether fine-tuning is the right approach for a new AI capability requirement.

Question	If YES	If NO
Can prompt engineering achieve required quality?	Use prompt engineering—skip fine-tuning	Proceed to next question
Is the task primarily knowledge retrieval of current information?	Use RAG—skip fine-tuning	Proceed to next question
Is consistent output format or style the primary goal?	Try few-shot prompting first	Proceed to next question
Is task-specific performance gain demonstrated in evaluation?	Fine-tune is justified	Re-examine task definition
Is VRAM / compute limited (< 24GB GPU)?	Use LoRA/PEFT	Consider full fine-tuning
Are multiple use cases required from same base?	Use separate LoRA adapters per use case	Single full fine-tune acceptable
Is general capability preservation critical?	Use LoRA or mixed data with low LR	Full fine-tune with monitoring

Note: Work through these questions in order. Fine-tuning should be a considered decision, not the default first approach for every AI capability gap.

Knowledge Check

Question 1: A team fine-tunes a general-purpose LLM on customer service data. Post-fine-tuning, task-specific performance improves by 18% but performance on a general language benchmark drops 14%. What has most likely occurred?

- A. The fine-tuning data was too high quality, causing the model to over-specialise
- **B. Catastrophic forgetting—the fine-tuning overwrote pre-trained weights encoding general capabilities**
- C. The model's context window was reduced during fine-tuning
- D. The instruction tuning stage was skipped during fine-tuning

Correct Answer: B. Catastrophic forgetting—the fine-tuning overwrote pre-trained weights encoding general capabilities

Explanation: Catastrophic forgetting occurs when fine-tuning gradients overwrite general pre-trained knowledge. The characteristic pattern is exactly this: task-specific improvement with general capability degradation. Mitigation: lower learning rate, LoRA, or adding general data to the fine-tuning mix.

Question 2: A startup wants to fine-tune Llama 3 70B to produce outputs in a specific brand writing style. They have a single 24GB GPU available. Which approach is feasible?

- A. Full fine-tuning—70B parameter models fit in 24GB with standard precision
- **B. LoRA with 4-bit quantisation (QLoRA)—fine-tuning only adapter parameters while keeping the base model quantised**
- C. Pre-training a new model from scratch on brand content
- D. RLHF—this is the correct approach for style adaptation

Correct Answer: B. LoRA with 4-bit quantisation (QLoRA)—fine-tuning only adapter parameters while keeping the base model quantised

Explanation: Full fine-tuning of 70B parameters requires ~140GB+ VRAM in half-precision—impossible on 24GB. QLoRA quantises the base model to 4-bit (fitting in ~35GB), then trains only small LoRA adapter layers in 16-bit—feasible on a single consumer GPU.

Question 3: An organisation needs an AI assistant to answer customer queries that include information about product releases from the last 30 days. What is the most appropriate technical approach?

- A. Fine-tune the model on a new dataset every 30 days to update its knowledge
- B. Use a larger pre-trained model with a more recent knowledge cutoff
- **C. Implement RAG with a database of recent product information, using the pre-trained model for reasoning and generation**
- D. Increase the model's temperature to allow more creative speculation about recent events

Correct Answer: C. Implement RAG with a database of recent product information, using the pre-trained model for reasoning and generation

Explanation: Continuously fine-tuning every 30 days is expensive and risks catastrophic forgetting. RAG retrieves current product information at query time and provides it as context to the model—giving accurate, current answers without retraining.

Reflection Questions

Take time to consider the following questions before the next session:

1. LoRA demonstrates that a model's behaviour can be substantially modified by training a tiny fraction of its parameters. What does this suggest about how pre-trained models store and organise knowledge?
2. RLHF uses human preference ratings to align model outputs with human values. What are the limitations of using human feedback as the training signal for alignment?
3. The availability of powerful open-weight models (Llama, Mistral) combined with accessible fine-tuning (LoRA) has democratised AI model development. What are the implications for AI safety and governance?
4. Catastrophic forgetting is a fundamental challenge in continual learning systems. Beyond mixing in general data, what architectural or training approaches might more robustly prevent it?
5. If fine-tuning on domain-specific data consistently outperforms general models on specific tasks, what does this suggest about the role of specialisation vs. generalisation in enterprise AI strategy?

Key Takeaways

- Fine-tuning adapts a pre-trained model to specific tasks, domains, or behaviours using task-specific datasets at a fraction of pre-training compute cost.
- Full fine-tuning updates all parameters—highest performance ceiling but requires significant memory and risks catastrophic forgetting.
- LoRA/PEFT trains only small adapter layers (~0.1–1% of parameters)—feasible on consumer hardware, enables multi-adapter deployment, reduces forgetting risk.
- Instruction tuning (SFT) and RLHF transform a text-completion base model into an instruction-following assistant—this is why commercial models respond helpfully rather than completing text.
- Catastrophic forgetting—loss of general capabilities during narrow fine-tuning—is mitigated by low learning rates, LoRA, and mixing general data into the fine-tuning corpus.
- Fine-tuning decision order: prompt engineering → RAG → LoRA fine-tuning → full fine-tuning. Escalate only when the simpler approach fails to meet quality requirements.

Next Actions

Complete at least two of the following before the next lecture:

6. Run a LoRA fine-tuning tutorial on a small open-weight model (e.g., using Hugging Face PEFT library with Llama 3 8B) to understand the practical setup process and parameter choices.
7. Compare Llama 3 8B base vs. Llama 3 8B Instruct on the same prompt to observe the practical difference instruction tuning makes to response quality and format.

8. Identify one internal use case where fine-tuning a general model on company-specific data would measurably improve output quality vs. prompting—document the training data you would need to collect.
9. Read the LoRA paper abstract ('LoRA: Low-Rank Adaptation of Large Language Models', Hu et al. 2021) to understand the mathematical basis of the rank-decomposition approach.